

Лабораторная работа № 1.

Основы тестирования серверных приложений

Тема: Unit tests, Arrange–Act–Assert, assertions и встроенный Node.js test runner

Цель: Освоить базовые принципы автоматического тестирования backend: выделить тестируемую бизнес-логику Internet Shop, написать позитивные, негативные и граничные unit tests через node:test и node:assert/strict и запускать test suite одной командой.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. В этой лабораторной Express server и PostgreSQL запускать для тестов не требуется: мы выделим небольшие чистые функции бизнес-логики и проверим их изолированно.

Почему используем node:test

Вводная лабораторная использует встроенный test runner Node.js и node:assert/strict. На следующем занятии те же принципы будут перенесены в Jest.

1. Создайте структуру тестов

Структура проекта

```
express-shop/  
  services/  
    orderMath.js  
  tests/  
    orderMath.test.js  
  package.json
```

2. Создайте тестируемую функцию расчёта заказа

services/orderMath.js

```
export function calculateOrderTotal(items) {  
  if (!Array.isArray(items)) {  
    throw new TypeError("items must be an array");  
  }  
  
  return items.reduce((sum, item) => {  
    if (!Number.isInteger(item.price) || item.price <= 0) {  
      throw new RangeError("price must be a positive integer");  
    }  
  
    if (  
      !Number.isInteger(item.quantity) ||  
      item.quantity < 1 ||  
      item.quantity > 100  
    ) {  
      throw new RangeError("quantity must be 1..100");  
    }  
  }, 0);  
}
```

```
    return sum + item.price * item.quantity;
  }, 0);
}
```

Функция не обращается к Express, Prisma или PostgreSQL. Поэтому её удобно проверять как unit.

3. Создайте первый test

```
tests/orderMath.test.js

import test from "node:test";
import assert from "node:assert/strict";
import { calculateOrderTotal } from "../services/orderMath.js";

test("calculates total for several items", () => {
  // Arrange
  const items = [
    { price: 10000, quantity: 2 },
    { price: 5000, quantity: 1 }
  ];

  // Act
  const total = calculateOrderTotal(items);

  // Assert
  assert.strictEqual(total, 25000);
});
```

В test case явно видны три части: Arrange, Act и Assert.

4. Запустите test

```
PowerShell

node --test tests/orderMath.test.js
```

Test должен завершиться успешно. Обратите внимание на количество выполненных tests и итоговый pass/fail status.

5. Добавьте npm script

```
package.json

{
  "scripts": {
    "test": "node --test",
    "test:watch": "node --test --watch"
  }
}
```

```
PowerShell

npm test
```

`node --test` автоматически запускает test files, найденные test runner.

6. Проведите эксперимент с failing test

1. Во временном варианте измените expected result с `25000` на `25001`.
2. Запустите `npm test`.
3. Найдите сообщение `assertion failure`.
4. Сравните `actual` и `expected`.
5. Верните expected result `25000`.
6. Повторите запуск и убедитесь, что test снова проходит.

Test должен уметь падать

Если test остаётся зелёным при заведомо неправильном expected result, такой test не проверяет нужное поведение.

7. Добавьте test для одного товара

```
tests/orderMath.test.js

test("calculates total for one item", () => {
  const items = [
    { price: 42000, quantity: 3 }
  ];

  const total = calculateOrderTotal(items);

  assert.strictEqual(total, 126000);
});
```

8. Добавьте test для пустого массива

```
tests/orderMath.test.js

test("returns zero for empty items array", () => {
  const total = calculateOrderTotal([]);

  assert.strictEqual(total, 0);
});
```

Этот test фиксирует контракт функции: пустой набор позиций имеет total `0`.

9. Проверьте нижнюю границу quantity

```
tests/orderMath.test.js

test("accepts quantity equal to one", () => {
  const total = calculateOrderTotal([
    { price: 15000, quantity: 1 }
  ]);

  assert.strictEqual(total, 15000);
});
```

```
});
```

10. Проверьте верхнюю границу quantity

tests/orderMath.test.js

```
test("accepts quantity equal to one hundred", () => {  
  const total = calculateOrderTotal([  
    { price: 1000, quantity: 100 }  
  ]);  
  
  assert.strictEqual(total, 100000);  
});
```

11. Добавьте negative test для quantity = 0

tests/orderMath.test.js

```
test("rejects quantity equal to zero", () => {  
  assert.throws(  
    () => calculateOrderTotal([  
      { price: 10000, quantity: 0 }  
    ]),  
    /quantity/  
  );  
});
```

12. Добавьте negative test выше верхней границы

tests/orderMath.test.js

```
test("rejects quantity greater than one hundred", () => {  
  assert.throws(  
    () => calculateOrderTotal([  
      { price: 10000, quantity: 101 }  
    ]),  
    /quantity/  
  );  
});
```

13. Проверьте некорректную цену

tests/orderMath.test.js

```
test("rejects non-positive price", () => {  
  assert.throws(  
    () => calculateOrderTotal([  
      { price: 0, quantity: 1 }  
    ]),  
    /price/  
  );  
});
```

14. Проверьте неправильный тип аргумента

```
tests/orderMath.test.js

test("rejects non-array input", () => {
  assert.throws(
    () => calculateOrderTotal(null),
    /items/
  );
});
```

15. Добавьте вторую бизнес-функцию

```
services/orderMath.js

export function canCancelOrder(status) {
  return status === "new";
}
```

Зафиксируем простое бизнес-правило: пользователь может отменить заказ только в статусе `new`.

16. Протестируйте `canCancelOrder()`

```
tests/orderMath.test.js

import {
  calculateOrderTotal,
  canCancelOrder
} from "../services/orderMath.js";

test("allows cancellation for new order", () => {
  assert.strictEqual(canCancelOrder("new"), true);
});

test("does not allow cancellation for paid order", () => {
  assert.strictEqual(canCancelOrder("paid"), false);
});

test("does not allow cancellation for shipped order", () => {
  assert.strictEqual(canCancelOrder("shipped"), false);
});
```

17. Запустите весь test suite

```
PowerShell

npm test
```

Все test cases должны выполняться независимо и давать одинаковый результат при повторном запуске.

18. Проверьте watch mode

```
PowerShell
```

```
npm run test:watch
```

Измените `orderMath.js` и посмотрите, как test runner повторно запускает tests после изменения файлов.

19. Классифицируйте созданные tests

Test	Уровень	Причина
<code>calculateOrderTotal()</code>	Unit	Проверяется одна чистая функция
<code>canCancelOrder()</code>	Unit	Нет HTTP, Prisma и DB
<code>POST /api/orders</code>	Пока не реализован	Будет integration test в следующих лекциях

20. Самостоятельное задание

- Добавьте test для двух позиций с quantity `100`.
- Добавьте test для отрицательной price.
- Добавьте test для дробной quantity.
- Добавьте test для status `cancelled` в `canCancelOrder()`.
- Придумайте ещё один boundary case.
- Запустите test несколько раз и убедитесь в deterministic result.
- Намеренно сломайте одну business function и убедитесь, что соответствующий test падает.
- Восстановите правильную реализацию и получите полностью зелёный test suite.

Требования к результату

- Создан каталог `tests`.
- Создан `orderMath.test.js`.
- Используются `node:test` и `node:assert/strict`.
- В `package.json` есть script `test`.
- Test cases используют понятные имена.
- Есть позитивные tests.
- Есть negative tests.
- Есть boundary tests.
- Используется `assert.strictEqual()`.
- Используется `assert.throws()`.
- Tests не используют production database.
- Повторный запуск даёт одинаковый результат.
- Все tests проходят после завершения работы.

Критерии оценивания

№	Критерий
---	----------

№	Критерий
1	Создан tests/orderMath.test.js
2	npm test запускает test suite
3	Есть test расчёта нескольких позиций
4	Есть test для одного item
5	Есть test пустого массива
6	Проверена нижняя boundary quantity
7	Проверена верхняя boundary quantity
8	quantity = 0 приводит к ожидаемой error
9	quantity = 101 приводит к ожидаемой error
10	Некорректная price приводит к ожидаемой error
11	Используется assert.strictEqual()
12	Используется assert.throws()
13	canCancelOrder() покрыта несколькими scenarios
14	Tests являются изолированными и deterministic
15	Студент объясняет различия unit, integration и E2E tests